

Neural Cadence:

Modeling Affective Responses to Music with fMRI Data

Cecilia Kuang, Josephine Nwaugha, Sophie On, Sydney Liu, Theo Benhabiles

Department of Cognitive Sciences, University of California, Irvine

COGS 108: Neural Analytics

Dr. Peters

December 12th, 2025

CrediT Style Contributions

Author	Data Curation	Formal Analysis	Funding Acquisition	Investigation	Methodology	Project Administration	Resources	Software	Supervision	Visualization	Writing-Original Draft	Writing-Review & Editing
Cecilia Kuang	X	X	N/A	X	X	X	X	X		X		X
Josephine Nwaughah	X		N/A	X				X				X
Sophie On	X		N/A	X			X	X				X
Sydney Liu	X	X	N/A	X	X		X	X		X		X
Theo Benhabiles			N/A	X					X		X	X

Abstract

How does listening to either classical music or artificial music influence brain function in areas related to hearing and motivation? We were interested in determining if the way people think about music can also influence the extent to which it motivates individuals to move or experience affective engagement. We used data collected by Daly and colleagues (2019) that consisted of fMRI measurements from two subjects in tasks where they had to listen to classical music, synthetic music, and washout stimuli. We used a region-of-interest (ROI) approach to analyze the fMRI data and focus our analysis on auditory cortex and subcortical structures involved in rewarding behavior or motivation (nucleus accumbens). In Nilearn, we created a multi-run General Linear Model (GLM) by combining all of the individual music clips together to create a single time series model. The expected output included beta maps, contrasts and z-maps of the whole brain, showing the difference in activation between the conditions. Neurosynth brain maps were used to mask the data and draw correlations between activity in our ROIs for subject 1's classical music data. Despite not having more than two subjects preprocessed and only about one analyzed for the classical task, we were able to draw a correlation between our regions of interest during the listening task.

Introduction

Our project initially aimed to examine how music influences motivation and physical performance, specifically in the context of running. Many people listen to music while exercising, and anecdotal reports suggest that music can enhance endurance, regulate pacing, and increase motivation. We were interested in whether distinct genres (rock, edm, pop, etc.), tempos, or beats per minute (BPM) produce measurable differences in neural activity during running. Ideally, this question would be addressed using EEG data collected during rhythmic movement, paired with time-frequency or decoding analyses. However, we quickly discovered that publicly available neuroimaging datasets combining music listening with running or movement tasks were extremely limited, making our original research question infeasible within the constraints of the course. To preserve the core theme of understanding how music shapes internal states such as motivation and engagement, we decided to shift our focus while preserving the underlying conceptual theme. This shift ultimately led us to explore an alternative dataset that still allowed us to investigate neural responses to different types of music.

Preprocessing Subject 1: Sophie On

Starting off with preprocessing: we attempted to divide and conquer by tackling two different subjects, with each subject being assigned to a separate student. For subject 1, I used fMRIPrep through Neurodesk and Docker to preprocess the data. One of the first issues I ran into was difficulty downloading the necessary files off of OpenNeuro—the provided download buttons and methods on the website were not working, so I had to download the files individually and reconstruct the correct folder format in order to have fMRIPrep correctly read the files. Fortunately, because we were provided with tutorial files in the class initially, I

essentially had a “template” that I could base my own work off of. I also utilized the .ipynb file provided in class, tweaking the code slightly so that the file paths were correct (for reading data and putting the output files in the correct folders).

Before attempting to run the code, I wanted to try and avoid the issues I had the first time I ran fMRIPrep—my code had run for a week before it finally finished, and despite not running into any errors along the way, I wished to speed up the process. Because of this, I went through the Docker settings and increased the resources available, upping the central processing unit (CPU), memory, and swap limit. However, I was hesitant on increasing the memory limit to the max, because I believed that it would cause issues when I was attempting to use my laptop for other work while the code was running. I was concerned that Docker would attempt to use up *all* the memory on my laptop, which could potentially make my computer run slower while doing classwork or taking notes during lectures. When tweaking the code in Neurodesk, despite increasing the CPU threads used, I also left the memory limit at 10 gigabytes. Due to this self-limitation, it’s possible that my code ran a bit slower than its max potential.

Unfortunately, I experienced the same issue that I had the first time I ran fMRIPrep: my code was taking an excessively long time to run! Even worse, fMRIPrep had failed and ran into an error after about 6 days of nonstop processing. Due to time limitations, we believed it best to just take the results that we had and focus on the next steps of the project, as we did get a decent amount of files from this initial run-through.

As for what went wrong, I initially believed that the error was a result of insufficient memory space. I consulted ChatGPT and briefly searched online for matches to my errors, and everything seemed to make sense—the process had self-terminated, and it made sense that it

would have to cancel the processing if it was unable to run any further due to limits on resources. I also learnt afterwards that increasing CPU threads (like I had done) led to diminishing returns on how much it would speed up the preprocessing time, and it would also increase the memory usage/requirement. After realizing this, I assumed that memory must've been the root of the issue.

However, upon further inspection, I had noticed that fMRIPrep ran into an `nipype.workflow` error three times before fully cancelling itself. I had difficulty finding the crashfiles immediately after the code had failed, as I was likely looking in the wrong folder location. Fortunately, I was able to find the actual crash logs, and after reading them and asking ChatGPT for some clarification, I now believe that the *actual* cause of failure might've been a corrupted file.

Figure 1

Snippet of Crash Log 1

```
File "/opt/conda/envs/fmriprep/lib/python3.11/site-packages/nibabel/arrayproxy.py",
line 457, in __array__
    arr = self._get_scaled(dtype=dtype, slicer=())
          ~~~~~
File "/opt/conda/envs/fmriprep/lib/python3.11/site-packages/nibabel/arrayproxy.py",
line 424, in _get_scaled
    scaled = apply_read_scaling(self._get_unscaled(slicer=slicer), scl_slope, scl_inter)
              ~~~~~
File "/opt/conda/envs/fmriprep/lib/python3.11/site-packages/nibabel/arrayproxy.py",
line 394, in _get_unscaled
    return array_from_file(
           ~~~~~
File "/opt/conda/envs/fmriprep/lib/python3.11/site-packages/nibabel/volumeutils.py",
line 472, in array_from_file
    raise OSError(
OSError: Expected 82747392 bytes, got 64556489 bytes from object
- could the file be damaged?
```

As seen with Figure 1, the expected file size did not align with the file size read. Given that it's stated in plain text in the crash log, my only assumption is that this *must* be the cause of my error. However, I'm unsure if the corrupted file is the `volumeutils.py`, or the original input file

at the top of the crash log, which is one of the files downloaded from the original dataset.

Assuming that the error originated from attempting to read the original data file, something must've gone wrong while downloading from OpenNeuro. Yet, when looking at the file size of my downloaded file and comparing it with the original file, they appear to have the same file size, which conflicts with what's stated in the crash log. ChatGPT also provided me with code to run that could apparently check the validity of the file I downloaded.

Figure 2

Error Confirming a Potential Corruption Issue

```
File /opt/conda/lib/python3.12/gzip.py:564, in _GzipReader.read(self, size)
    562         break
    563     if buf == b'':
--> 564         raise EOFError("Compressed file ended before the "
    565                          "end-of-stream marker was reached")
    567 self._crc = zlib.crc32(uncompress, self._crc)
    568 self._stream_size += len(uncompress)

EOFError: Compressed file ended before the end-of-stream marker was reached
```

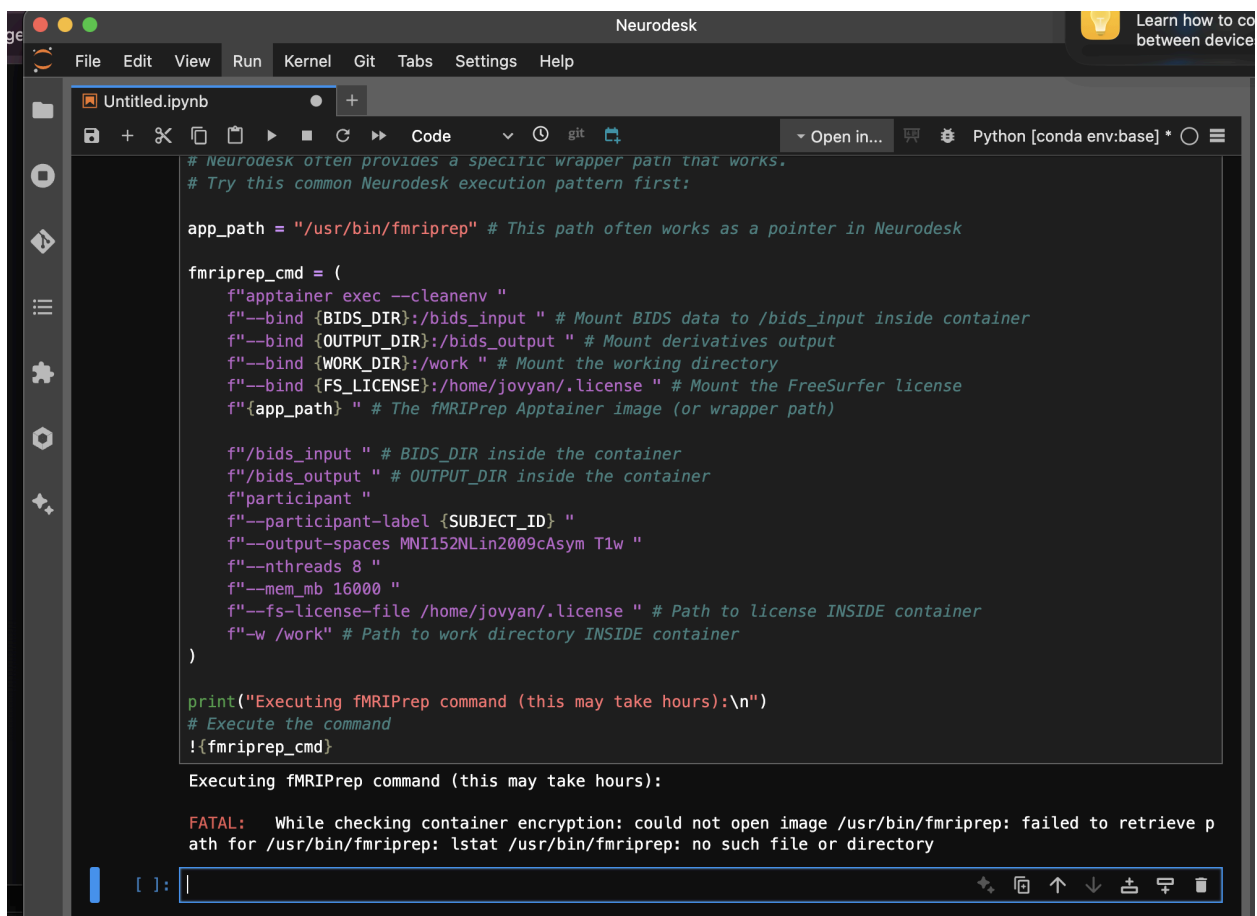
After running the file through that code, it did appear to confirm that there was some kind of corruption issue, as stated by the “compressed file ended before the end-of-stream marker was reached”. Ultimately, while I was unable to preprocess subject 1 fully through fMRIPrep without any errors, I believe I was able to get relatively far and output a decent amount of usable data. This was also a good learning experience in tinkering with AI chatbots, code, and laptop limitations and resources.

Preprocessing Subject 2: Josephine Nwaughu

My primary responsibility for the project involved the preprocessing of fMRI data for Subject 2 using the industry-standard fMRIPrep pipeline. This step was necessary to correct for motion artifacts, distortions, and standardize anatomical alignment. Still, it was complicated by persistent software failures that ultimately required over ten days of concentrated troubleshooting. Initial attempts to run the fMRIPrep Docker container via high-level integrated environments, specifically NeuroDesk and VS Code, were repeatedly unsuccessful and time-consuming to say the least.

Figure 3:

Initial container Docker failure



```

Neurodesk
File Edit View Run Kernel Git Tabs Settings Help

Untitled.ipynb
+
Code
Open in... Python [conda env:base] *

# Neurodesk often provides a specific wrapper path that works.
# Try this common Neurodesk execution pattern first:

app_path = "/usr/bin/fmriprep" # This path often works as a pointer in Neurodesk

fmriprep_cmd = (
    f"apptainer exec --cleanenv "
    f"--bind {BIDS_DIR}:/bids_input " # Mount BIDS data to /bids_input inside container
    f"--bind {OUTPUT_DIR}:/bids_output " # Mount derivatives output
    f"--bind {WORK_DIR}:/work " # Mount the working directory
    f"--bind {FS_LICENSE}:/home/jovyan/.license " # Mount the FreeSurfer license
    f"{app_path} " # The fMRIPrep Apptainer image (or wrapper path)

    f"/bids_input " # BIDS_DIR inside the container
    f"/bids_output " # OUTPUT_DIR inside the container
    f"participant "
    f"--participant-label {SUBJECT_ID} "
    f"--output-spaces MNI152NLin2009cAsym T1w "
    f"--nthreads 8 "
    f"--mem_mb 16000 "
    f"--fs-license-file /home/jovyan/.license " # Path to license INSIDE container
    f"-w /work" # Path to work directory INSIDE container
)

print("Executing fMRIPrep command (this may take hours):\n")
# Execute the command
!{fmriprep_cmd}

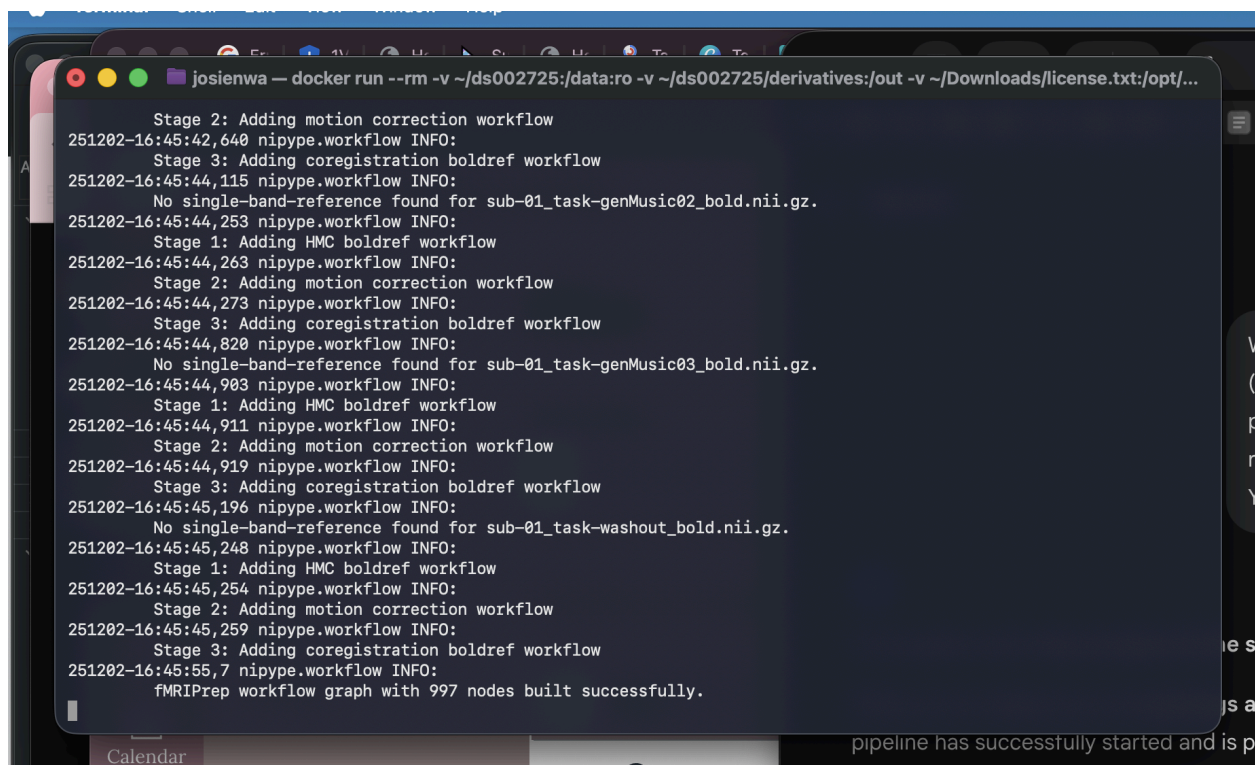
Executing fMRIPrep command (this may take hours):

FATAL: While checking container encryption: could not open image /usr/bin/fmriprep: failed to retrieve p
ath for /usr/bin/fmriprep: lstat /usr/bin/fmriprep: no such file or directory
  
```

The most significant barrier encountered was a FATAL error about a missing file, which prevented data access for over a week and a half. Further investigation determined the issue was not due to resource constraints like memory (as the laptop was new and data was wiped clean), but rather problems persistent with Docker container authentication and volume mounting, and/or issues rooted in network failures and permission denials. To successfully launch the preprocessing job, I abandoned the failing scripts and adopted a bare-metal Docker CLI (Command Line Interface) approach via the Mac Terminal.

Figure 4:

Successful fMRIPrep CLI Execution



```
josienwa — docker run --rm -v ~/ds002725:/data:ro -v ~/ds002725/derivatives:/out -v ~/Downloads/license.txt:/opt/...

Stage 2: Adding motion correction workflow
251202-16:45:42,640 nipype.workflow INFO:
Stage 3: Adding coregistration boldref workflow
251202-16:45:44,115 nipype.workflow INFO:
No single-band-reference found for sub-01_task-genMusic02_bold.nii.gz.
251202-16:45:44,253 nipype.workflow INFO:
Stage 1: Adding HMC boldref workflow
251202-16:45:44,263 nipype.workflow INFO:
Stage 2: Adding motion correction workflow
251202-16:45:44,273 nipype.workflow INFO:
Stage 3: Adding coregistration boldref workflow
251202-16:45:44,820 nipype.workflow INFO:
No single-band-reference found for sub-01_task-genMusic03_bold.nii.gz.
251202-16:45:44,903 nipype.workflow INFO:
Stage 1: Adding HMC boldref workflow
251202-16:45:44,911 nipype.workflow INFO:
Stage 2: Adding motion correction workflow
251202-16:45:44,919 nipype.workflow INFO:
Stage 3: Adding coregistration boldref workflow
251202-16:45:45,196 nipype.workflow INFO:
No single-band-reference found for sub-01_task-washout_bold.nii.gz.
251202-16:45:45,248 nipype.workflow INFO:
Stage 1: Adding HMC boldref workflow
251202-16:45:45,254 nipype.workflow INFO:
Stage 2: Adding motion correction workflow
251202-16:45:45,259 nipype.workflow INFO:
Stage 3: Adding coregistration boldref workflow
251202-16:45:55,7 nipype.workflow INFO:
fMRIPrep workflow graph with 997 nodes built successfully.
```

pipeline has successfully started and is p

This resolution, which represented the successful implementation after numerous setbacks, required two crucial fixes drawing on knowledge from external coding classes: executing a successful Docker login to resolve the pull access denied errors, and manually mounting the necessary FreeSurfer license path for container validation. Although the pipeline was successfully launched on November 30th and completed by December 1st, this extensive troubleshooting period contributed to significant time constraints for the team, resulting in the strategic trade-off of prioritizing Subject 1's already-available preprocessed data for the final analysis to ensure the project deadline was met. In all, this experience served as a highly valuable learning process in troubleshooting, resource management, and the notion to leverage knowledge from not only chatbots and coding websites, but also using old assignments and learning outcomes to solve specific problems in software.

Timing and regressor: Theo Benhabiles

As the timing and regressor lead on the project, my main responsibility was to identify the affective arousal ratings for each of the music clips used in the study and then link those ratings with the start and length of each fMRI run. Ideally, in an effective fMRI design, all of the music clips should appear in a single BOLD time series in order to allow the modeling of emotional intensity through parametric regressors. Therefore, my goal was to create the timing files and the regressors to make it possible to incorporate the time and emotion aspects into the group's General Linear Model (GLM) analysis. There were two major barriers preventing this work from being accomplished. First, the dataset did not contain the arousal ratings that had been

intended to accompany each of the stimuli; without the ratings, any attempts at modeling emotional intensity would have necessitated the assignment of subjective estimates to each of the music clips, which would not have represented a scientific method. Acquiring the correct arousal ratings from the researchers who created the dataset would have required some communication with the original researchers, however, this was outside of the scope of time allowed by the project.

The second and larger barrier to completing this work came about due to the design of the experiment. Rather than collecting all of the music clips within one unbroken scanning session, which is typically done in studies using fMRI, the researchers started and stopped the scanner after each music clip. Therefore, each of the individual music trials was recorded separately in a different run, resulting in each of the clips having their own unique baseline, drift, and scanner stabilization profile. As a result, the event files could not be utilized to represent the actual timing of each clip, and would require combining multiple, short runs, into a pseudo-continuous sequence, prior to creating the appropriate regressors. While technically possible, such an action would have introduced many confounds into the data and increased the amount of processing that would have been necessary to prepare the data for statistical analysis. The problems presented in this regard also paralleled the challenges that the group experienced when trying to develop a unified GLM in VS Code, further demonstrating how the structural format of the dataset hindered the use of timing based analyses. Given additional time, I would have been able to contact the original researchers in order to obtain the missing arousal ratings, to merge the segmented runs in an appropriate manner, and to produce the timing files and parametric regressors needed to model the effects of arousal driven neural responses properly.

General Linear Model (GLM) Analysis: Cecilia Kuang

To examine neural activation associated with different types of music, we employed a General Linear Model (GLM)-based approach. In fMRI research, the GLM is commonly used to model how the blood-oxygen-level-dependent (BOLD) signal in each voxel varies over time as a function of experimental conditions and nuisance variables. This framework allows for statistical estimation of condition-specific activation and enables direct contrast between conditions of interest. We chose a GLM approach because it is well-suited for identifying brain regions that respond differently to classical versus synthetic music and is widely used in task-based fMRI analyses. All preprocessing was performed using fMRIPrep, which provided preprocessed BOLD images in MNI space, brain masks, and confound regressors including motion parameters and framewise displacement. Initially, we planned to use event-based modeling; however, each fMRI run in the dataset consisted of a single continuous condition (classical music, synthetic music, or washout baseline), rendering the event files unsuitable for traditional trial-level GLM modeling. As a result, we adapted our analysis strategy to accommodate this limitation.

To enable statistical comparisons between conditions that occurred in separate runs, I concatenated all five runs into a single continuous fMRI time series. Then I constructed a unified design matrix that included condition regressors for classical and synthetic music, nuisance regressors derived from fMRIPrep confounds, and additional run-intercept regressors to account for run-specific baseline shifts. These run intercepts allowed us to model differences in scanner baseline and signal intensity across runs without conflating them with condition effects. A single first-level GLM was fit to the concatenated dataset using Nilearn's `FirstLevelModel`, and voxelwise contrast maps were computed to compare classical versus synthetic music as well as

music versus baseline. Statistical results were visualized using whole-brain z-maps, which were thresholded using false discovery rate (FDR) correction to control for multiple comparisons.

Figure 5 shows a portion of the Python code used to implement this model, including the fitting of the concatenated GLM and the manual construction of contrast vectors required by the absence of event-level regressors. We also utilized GitHub to share our notebooks and codes with each other.

Figure 5. Excerpt from the Python script implementing the multi-run GLM analysis.

```

glm_multirun.py 4 X Nilearn Template_Skeleton.ipynb
C:\Users> Cecilia > Downloads > glm_multirun.py > run_first_level_multirun
160 def run_first_level_multirun(project_root, subject, runs, tr, out_subdir_name=None):
175
176     print("Design matrix columns:", design_matrix.columns.tolist())
177
178     # Fit first level model
179     flm = FirstLevelModel(
180         t_r=tr,
181         slice_time_ref=0.5,
182         mask_img=mask_path,
183         hrf_model='glover',
184         drift_model='cosine',
185         high_pass=1/128,
186     )
187
188     print("Fitting model to concatenated runs...")
189     flm = flm.fit(fmri_img_concat, design_matrices=design_matrix, confounds=confounds_concat)
190
191     # Build contrasts
192     columns = list(design_matrix.columns)
193
194     def make_contrast(cond_pos, cond_neg=None):
195         # cond_pos/cnd_neg can be strings or list of strings
196         vec = np.zeros(len(columns))
197         if cond_pos is not None:
198             for name in (cond_pos if isinstance(cond_pos, (list, tuple)) else [cond_pos]):
199                 if name in columns:
200                     vec[columns.index(name)] = 1
201         if cond_neg is not None:
202             for name in (cond_neg if isinstance(cond_neg, (list, tuple)) else [cond_neg]):
203                 if name in columns:
204                     vec[columns.index(name)] = -1
205         return vec
206
207     # classical > synthetic
208     contrast_classical_gt_synth = make_contrast('classical', 'synthetic')
209     # synthetic > classical
210     contrast_synth_gt_classical = make_contrast('synthetic', 'classical')
211     # music (classical + synthetic) > baseline
212     contrast_music_vs_baseline = make_contrast(['classical', 'synthetic'], None)
213
214     print("Computing contrasts (z-maps)...")
215     z_music = flm.compute_contrast(contrast_music_vs_baseline, output_type='z_score')
216     z_class_vs_synth = flm.compute_contrast(contrast_classical_gt_synth, output_type='z_score')
217     z_synth_vs_class = flm.compute_contrast(contrast_synth_gt_classical, output_type='z_score')

```

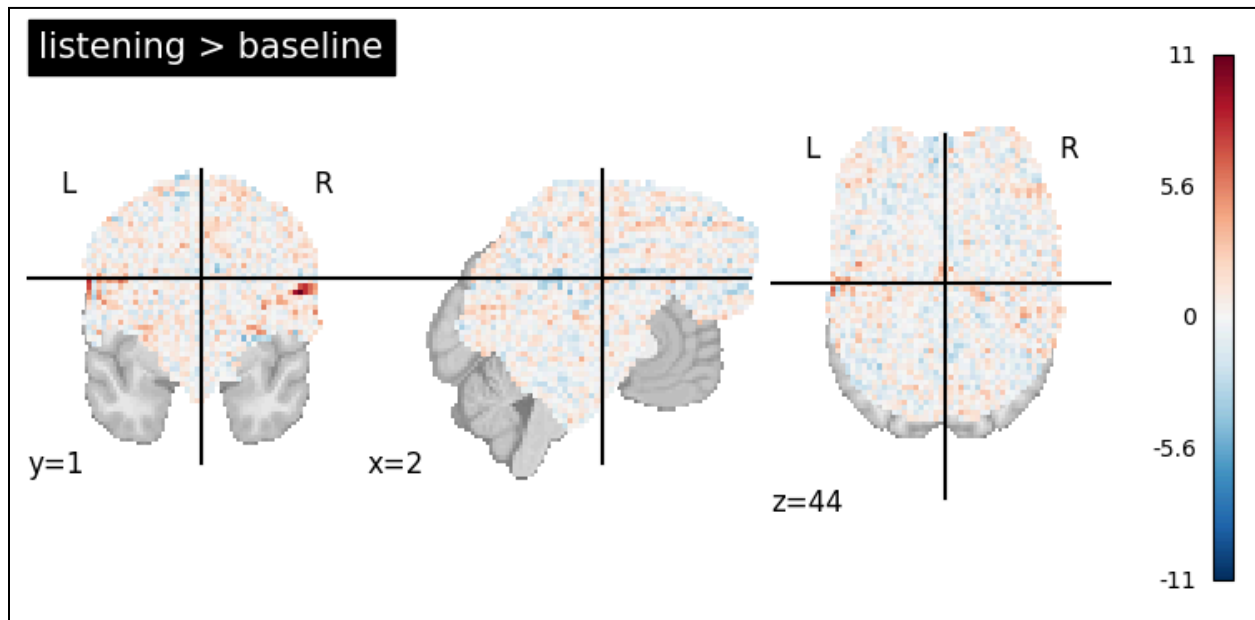
Due to time constraints, I was unable to complete the final GLM analysis on the preprocessed music data. However, prior to applying the GLM to the project dataset, I wanted to

ensure that the code works by using a standard auditory fMRI dataset provided by Nilearn. This dataset contains a clearly defined listening task and serves as a benchmark for confirming that the GLM implementation, contrast specification, and visualization procedures were working and would produce visuals. Using this dataset, I constructed a design matrix, fit a first-level GLM, and computed a contrast comparing listening versus baseline conditions.

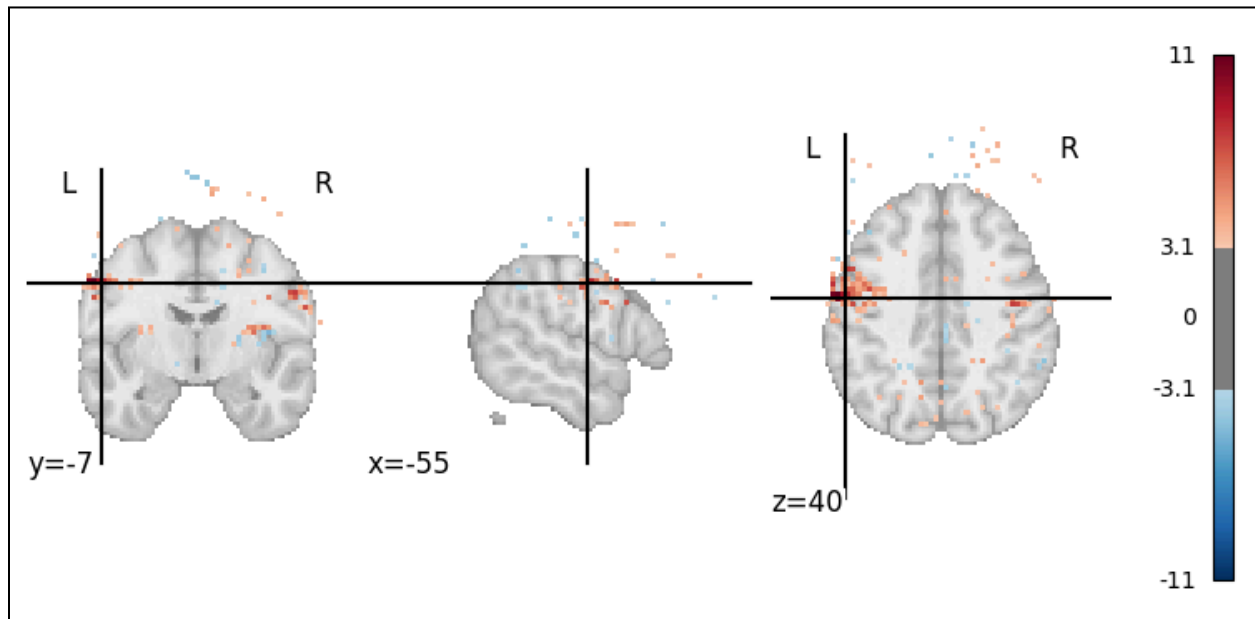
Figure 6 presents the resulting whole-brain z-maps from this sample dataset, proving that the GLM worked. The unthresholded z-map (A) shows widespread activation patterns, while the FDR-thresholded z-map (B) highlights statistically significant clusters. As expected, strong bilateral activation is observed in the superior temporal cortex, consistent with primary and secondary auditory regions. These results demonstrate that the GLM pipeline successfully captures neural responses to auditory stimulation and produces biologically plausible activation patterns. Although equivalent z-maps could not be generated for the classical versus synthetic music comparison using the project dataset, this demonstration supports the hypothesis that music robustly activates bilateral auditory cortex and suggests that differences between classical and synthetic music would likely manifest as subtle modulations within these auditory regions. I learned that GitHub Copilot could be extremely useful when we had to revamp our analyses and wish we had more time so I could try to have GitHub directly read our files, change pathways for me automatically, and then run the code for z-maps and contrasts with our actual dataset. It was definitely a learning experience and now I would know what to do better if I were to run an analysis again in the future.

Figure 6. Demonstration GLM results using the Nilearn SPM auditory dataset

A) *Unthresholded whole-brain z-map for the listening > baseline contrast, showing widespread activation patterns across the cortex.*



B) *The same contrast after false discovery rate (FDR) correction, highlighting significant clusters*



Building the Mask: Sydney Liu

Before getting into the process of creating a mask, what exactly is the goal of creating a mask and what even is a mask? The goal here is to create a mask that isolates voxels in our BOLD data so we can draw correlations between our regions of interest which are the neural centers for auditory processing and motivation. The template for the mask itself came from the brain maps made by the meta-analysis data from Neurosynth.

To begin the process of making things though, I started off using Gemini as my AI assistant. Since I didn't quite know what a mask was at this point, the first thing I had Gemini do was break down what I needed and how it was all going to be used in the code. This was a good way to figure out that I needed a tsv file and the coregistered BOLD MNI file from the preprocessed data. Alas, now we can begin to code. However, I didn't realize this at the time, but I started this particular journey with a fatal error: I built the code with Gemini in chunks. At first, this seems like a reasonable approach in case I have any blocks of code that might take a while to run on their own. This instead made the code more confusing and longer than necessary.

Initially, I was coding in Google Colab since making the code in a collaborative environment made sense at the time, since our roles weren't completely clear cut at that point in the project. This approach though just added fuel to the fire that was growing as I created more chunks of code from Gemini. Reason being the way colab disconnects to its Python server that makes the files I put in its directory disappear every time I want to reconnect. This was mainly just frustrating since occasionally the file paths I put into the code became incorrect. So to avoid further frustration from trivial details not working, I switched to Visual Studio Code.

Thankfully, Visual Studio Code (VS code) has a copilot built into the program, so I thought it would be smooth sailing from there on out. I didn't start from scratch though. I just

pasted my colab code into VS code and changed the file paths to start. As one might expect, there were some errors. Things like memory issues and long run times kept coming up no matter how I tweaked the code here and there. Instead of completely scrapping the whole thing when things were going wrong though, I tried using copilot to figure out the issue and possible solutions. Considering I had very little to no experience using libraries like Nilearn, I couldn't trust myself to try to tweak the code in meaningful ways on my own. But alas, that is why it is called "copilot". This AI tool and I tried for days to make the Colab code work, even trying to create progress report files to show me that something is happening at all, but as I have been foreshadowing, eventually I decided to start anew. When using copilot though, I believe I was making an error in how I was talking to it since I may have been too vague in my prompts and questions as I likely had been with Gemini.

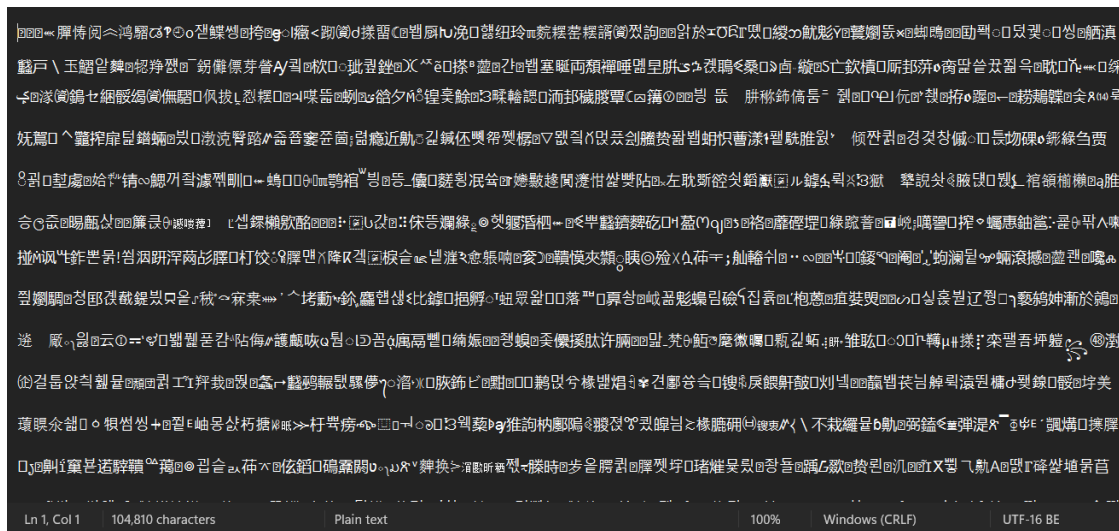


Figure 7. Example of nonsense output from progress my attempt at creating progress reports

With a clean slate now, I decided to be “painfully specific” with my prompt. I included specific file names and what to use them for as well as specifying the path from creating the

mask to what I wanted as an output. After creating the most engineered prompt I have ever created, copilot came through for me by creating a far more concise and simple jupyter notebook. This code didn't quite give me the brain images I was aiming for, but it was able to produce a graph that illustrates the correlation between activity in the auditory cortex and nucleus accumbens (our regions of interest). In the scatter plot that was generated, it's quite clear that we can create a line of best fit that describes the data well. We were able to find a correlation of about 0.6 in the end which leads us to think there's a strong correlation between the activity in the brain regions dealing with auditory processing and motivation.

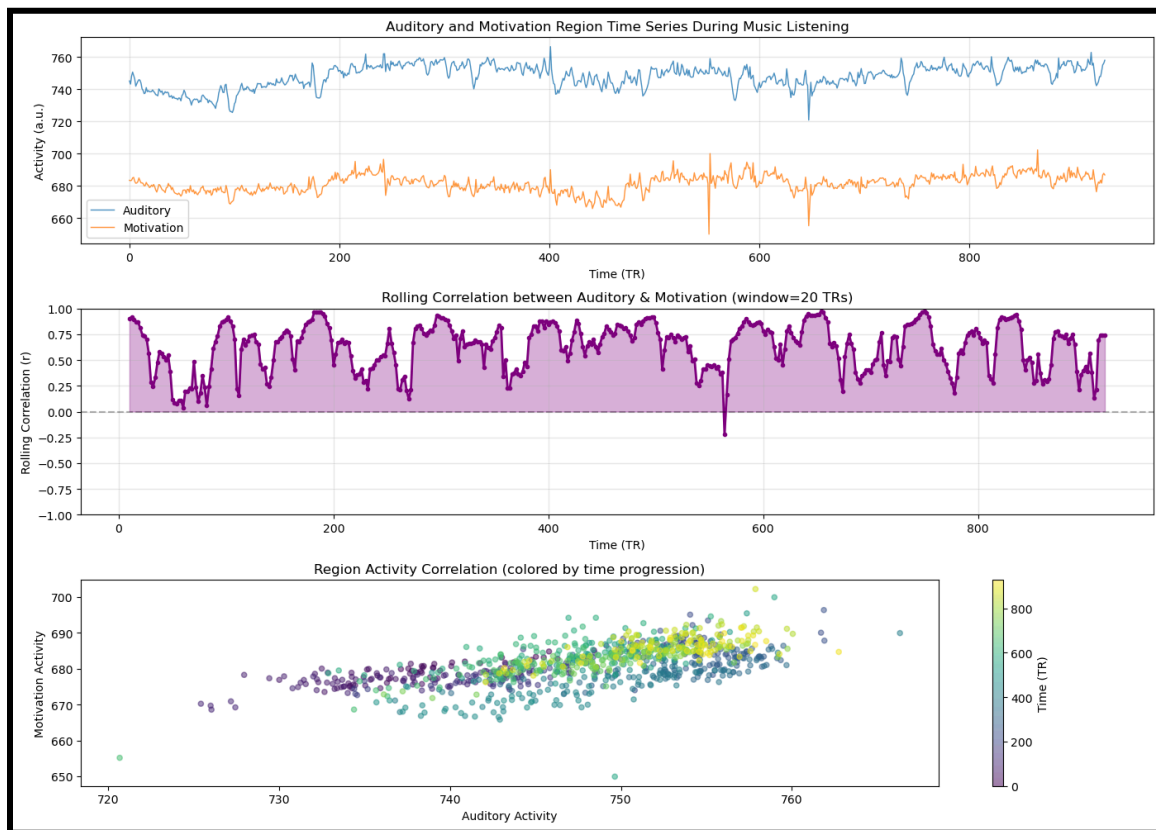


Figure 8. Output from Visual Studio Code final copilot generated code from mask files

With the time constraints at hand though, the masking process was only applied to one subject with the classical music task data. With more time and ability to work with more subjects, we could likely predict an even stronger correlation between the regions of interest.

What If We Had More Time?

Preprocessing

Given that time was one of our biggest constraints for preprocessing, we likely would've managed to finish processing subject 1 and 2's data if fMRIPrep was running for longer. Any of the issues we encountered would be able to be worked through, and if fMRIPrep were to run into errors again, we could either fix them as we go and rerun the code, or we could also have sunk some time into learning a different method of preprocessing (like MATLAB).

It would also be preferable to spend the time setting up Docker and Neurodesk on a different, more powerful computer, which could alleviate issues with not having enough resources for memory, container issues, or storage space. From there, we could run fMRIPrep faster, potentially allowing us to preprocess data for more than just two subjects.

Timing and Regressor

If we had more time, we would have reached out to the researchers of the original study to try and get the arousal ratings for each music clip, which could take weeks for a response. If they refused, or did not respond, we would have conducted a survey, asking people to rank each music clip on a numerical scale from 1 to 10. More time would have also allowed us to piece together the separate runs while accounting for baseline shifts through confound regression for better results.

GLM and Mask

If we had more time, we would process more subjects and data from both the synthetic music task and washout. That way, we would be able to create a baseline using the washout (nonsense stimuli), and we would be able to explore the condition of synthetic music and its effect on the correlation between our regions of interest.

In the creation of our GLM, we would apply the contrasts we had created between conditions. We would likely have to double check that the conditions being contrasted are anticorrelated so that the results make sense when we check the activity in any given region.

In creating the mask, going back and understanding each step of the mask creation process would be a good start to refining the code we already have and being able to output a more interactive brain map like those shown in class.

PPI Connectivity

If additional time and data had been available, we would have extended the current analysis by incorporating the original affective arousal ratings referenced in the accompanying study. Access to these ratings would have enabled a more nuanced investigation of how emotional intensity modulates neural processing of music. Specifically, we would have implemented a psychophysiological interaction (PPI) analysis to examine task-dependent changes in functional connectivity between auditory regions and emotion-related areas such as the amygdala, ventromedial prefrontal cortex, and anterior cingulate cortex. PPI analyses are well suited for testing whether the relationship between brain regions varies as a function of psychological context, in this case different types of music and levels of emotional arousal.

In addition, the arousal ratings could have been incorporated into the GLM as a parametric modulator, allowing for the modeling of continuous variation in perceived emotional intensity rather than treating music conditions as categorical blocks. This approach would have enabled testing whether increases in arousal were associated with stronger activation in specific regions or altered connectivity patterns across the auditory–limbic network. Together, these analyses would move beyond identifying where music-related activation occurs to examining how emotional engagement dynamically shapes interactions between brain regions, providing a more comprehensive account of the neural mechanisms underlying music-induced motivation and affect. It would have been cool to incorporate this as we could have potentially tied it back to our original question and interest.

Conclusion

Even with the small amount of data we were able to process, our question did get answered. We were indeed able to find a correlation of about 0.6 between the brain regions that prioritize auditory input and motivation. Did we answer our question well though? Had we had more time, we could have answered our question with greater confidence. Having only fully processed about one subject makes the question of whether our results are up to individual differences or not. For future endeavors though, after fighting through our struggles, we have refined our art of prompt engineering with AI tools and are now more experienced in the realm of neural data processing. Also, knowing that our current tools and code work well for us, it is entirely possible for us to face the full scale of our question should we choose to in the future.

Works Cited

Daly, I., Williams, D., Hwang, F. et al. Electroencephalography reflects the activity of sub-cortical brain regions during approach-withdrawal behaviour while listening to music. *Sci Rep* 9, 9415 (2019). <https://doi.org/10.1038/s41598-019-45105-2>